



**TOBB ETÜ**  
Ekonomi ve Teknoloji Üniversitesi

**TOBB EKONOMİ VE TEKNOLOJİ ÜNİVERSİTESİ**  
Bilgisayar / Yapay Zekâ Mühendisliği  
**BİL 401**

## Project Status Report

A Simulated Flash-Sale Distributed Data Pipeline: Kafka, Spark, and Citus on Docker Compose

|                       |                                                        |
|-----------------------|--------------------------------------------------------|
| <b>Takım Üyeleri</b>  | Mete KILIÇ- 231401028<br>İsmail Emre YILDIZ- 221401006 |
| <b>Akademik Dönem</b> | 2025–2026 Yaz Dönemi                                   |

## Table of Contents

|                                                                        |          |
|------------------------------------------------------------------------|----------|
| <b>1. Abstract.....</b>                                                | <b>2</b> |
| <b>2. Related Work .....</b>                                           | <b>2</b> |
| <b>3. System Architecture (Proposed Implementation, Updated) .....</b> | <b>3</b> |
| <b>4. State of Data Collection .....</b>                               | <b>3</b> |
| <b>5. Status of Platforms and Systems .....</b>                        | <b>4</b> |
| <b>6. Installation Problems, Know-How, and Demo Runs .....</b>         | <b>4</b> |
| 6.1 The Bitnami problem.....                                           | 4        |
| 6.2 Citus inter-node authentication .....                              | 4        |
| 6.3 Grafana datasource provisioning .....                              | 5        |
| 6.4 Demo runs performed so far .....                                   | 5        |
| <b>7. Remaining Work Toward the Final Report .....</b>                 | <b>5</b> |

## 1. Abstract

This report documents the current status of our Systems Track project: a simulated “Flash Sale” data pipeline built around Apache Kafka, Apache Spark Structured Streaming, and Citus (distributed PostgreSQL), all orchestrated locally with Docker Compose. Since Checkpoint 1, the pipeline has gone from a design on paper to a system that has been running close to continuously on a development laptop for several days, processing tens of millions of synthetic clickstream events.

At this point we have, honestly, already done a fair amount of what we originally expected to leave for the final report stage – including fault-injection tests on all three core technologies and two separate rounds of horizontal scaling (Spark workers and Citus workers). Part of the reason is good progress, and part of it is that an unrelated infrastructure problem (a vendor removing the Docker images our original plan depended on, described in Section 6) forced us to rebuild a chunk of the deployment earlier than planned, which in turn pushed us to test it more thoroughly than we initially intended. This report summarizes the architecture as it stands today, the state of data generation, the health of each platform, and the practical lessons learned while getting everything to actually work together.

## 2. Related Work

The general pattern we are following – Kafka as an ingestion buffer feeding a stream processor that writes into a horizontally-scaled relational store – is a well-established one in industry, tracing back to Kafka’s original role as a commit log for decoupling producers and consumers. Spark Structured Streaming’s micro-batch model is a common way to combine batch-style logic (custom aggregation, conditional filtering, joins against reference data) with a continuously arriving stream; this is why we used its `foreachBatch` sink rather than the purely declarative streaming API, which would not have let us run the per-batch anomaly scoring and the upsert-style writes we needed.

Citus is a reasonably well-known PostgreSQL extension that turns a regular Postgres cluster into a sharded, horizontally-scalable database without giving up standard SQL, and it is used in production by several SaaS companies for exactly the kind of high-write, time-series-like workload we are simulating here (one coordinator routing queries, several workers each holding a slice of the data).

On the anomaly-detection side, our rule – flag any IP address responsible for more than N events inside a single micro-batch – is a deliberately simplified version of the rate-limiting and bot-scoring techniques typically discussed around ticket and sneaker resale bots, where real systems usually combine rate limiting with CAPTCHAs and behavioral scoring. We picked the simplest possible version on purpose, since the point of this project is the distributed systems engineering rather than the anomaly-detection algorithm itself. As it turned out (Section 6), this simplicity has a real, measurable downside under heavy load, which we think is actually a more useful result for the report than if the naive threshold had just worked perfectly the whole time.

### 3. System Architecture (Proposed Implementation, Updated)

The pipeline, as currently deployed, looks like this:

- A Python producer (kafka-python client) generates a synthetic e-commerce clickstream – user\_id, action, product\_id, timestamp, ip\_address – and cycles automatically between a NORMAL phase (~100 events/sec, no bots) and a FLASH\_SALE phase (~10,000 events/sec target, ~15% bot/scalper traffic concentrated on a small set of “hot” products and a small pool of IPs).
- A 3-broker Apache Kafka cluster in KRaft mode (no ZooKeeper) buffers the stream. The topic has 6 partitions, replication factor 3, and min.insync.replicas=2.
- Apache Spark Structured Streaming consumes the topic and, for every micro-batch, counts events per IP address; any IP above a configured threshold is written to an events\_flagged table with a reason code, everything else goes to events\_clean, and a per-product/action aggregate is upserted into events\_agg. A small pipeline\_metrics table records one row per batch (size, latency, flagged count, etc.) for monitoring.
- Citus (one coordinator, currently three workers – see Section 5) stores all of the above as sharded distributed tables, with the small metrics table kept as a regular coordinator-local table.
- Prometheus scrapes Kafka and Postgres exporters, and Grafana visualizes pipeline\_metrics and Kafka throughput on a single dashboard.

Everything is defined in one docker-compose.yml file (currently 17 services) and runs on a single Windows 11 laptop via WSL2 and Docker Desktop – deliberately not Kubernetes. We considered this briefly, but the project brief explicitly asks for something that runs on a student laptop, and adding a Kubernetes layer on top of an already-multi-container Compose setup would add a second orchestration model for no real benefit at this scale; Compose already gives us the “simulated distributed environment” the brief asks for.

One small but, we think, worth-mentioning design point: our Kafka topic has 6 partitions, and Spark’s read-side parallelism for a Kafka source is capped at one task per partition. We therefore tested Spark scaling up to 3 workers × 2 cores = 6 cores, which lines up with the partition count, rather than scaling further – past that point, more cores would not buy us more Kafka read parallelism, only some extra room on the aggregation/JDBC-write side. We treat this as a sensible, reasoned stopping point rather than an arbitrary one.

### 4. State of Data Collection

As stated in Checkpoint 1, we are not using a static or scraped dataset; the project generates its own synthetic clickstream. This part of the project is functionally complete: the generator produces realistic action-type distributions (mostly VIEW, some ADD\_TO\_CART, fewer PURCHASE for normal traffic; the opposite skew for bot traffic), supports both the automatic NORMAL/FLASH\_SALE cycle and a manual trigger for live demos, and has been running for days without needing to be restarted for correctness reasons.

One thing we did not expect to find here: during a stress test (Section 6) we discovered that our own single-process Python generator tops out at roughly 10,800–11,000 events/sec, regardless of how much higher we set the target rate. So in a sense, “data collection” for this project has its own scalability ceiling too, and it is on the producer side rather than anywhere in Kafka/Spark/Citus. We note this honestly here because it affects how we should phrase some of the later results – a few of our high-load batches were ultimately rate-limited by our own generator, not by the system under test.

## 5. Status of Platforms and Systems

| Component                         | Status                                                                                         |
|-----------------------------------|------------------------------------------------------------------------------------------------|
| Docker / Docker Compose           | Stable. Running on Docker Desktop (WSL2 backend), 17 services.                                 |
| Apache Kafka (3 brokers, KRaft)   | Healthy. Topic with 6 partitions / RF=3 / min.insync.replicas=2. Fault-tested (see Section 6). |
| Apache Spark (1 master + workers) | Healthy. Currently running with 3 workers (scaled up from 1 during testing).                   |
| Citus (1 coordinator + 3 workers) | Healthy. Started with 2 workers; a 3rd was added and shards rebalanced mid-project.            |
| Prometheus / Grafana              | Operational, after fixing an early datasource provisioning bug (Section 6).                    |
| Synthetic producer                | Operational, see Section 4 for its own throughput limit.                                       |

In short: every component we listed in Checkpoint 1 is installed, integrated, and has been observed processing real (synthetic) traffic, not just “started and left idle.”

## 6. Installation Problems, Know-How, and Demo Runs

### 6.1 The Bitnami problem

Our original docker-compose.yml used Bitnami’s prebuilt Kafka and Spark images, which is what most online tutorials still recommend. When we actually tried to bring the stack up on the deployment laptop, both pulls failed with “image not found.” It turned out that Broadcom (which now owns Bitnami) had removed almost all versioned tags from the free docker.io/bitnami namespace, leaving only a small paid “Secure Images” catalogue that does not even include Kafka or Spark for free users. This was not something we could have anticipated from our Checkpoint 1 plan, and it meant rebuilding two of our four core services’ images from scratch: we switched Kafka to the official apache/kafka image (which uses a different environment-variable naming scheme, so the KRaft configuration had to be rewritten) and built our own Spark image on top of a plain Python base image plus the official Apache Spark binary distribution. It cost us real time, but arguably left us with a deployment that depends on fewer third-party packaging decisions.

### 6.2 Citus inter-node authentication

After restarting the whole stack (e.g., after the laptop itself was restarted), the Citus coordinator would sometimes fail to talk to its own worker nodes with a “no password supplied” error, even though everything had worked minutes earlier. The cause was Citus’s internal node-to-node authentication, which is separate from the client-facing authentication we had already configured. The fix was to register an explicit password for the citus role at the Citus metadata level (the pg\_dist\_authinfo table), applied on every node. This is now baked into our coordinator bootstrap script so it survives restarts.

## 6.3 Grafana datasource provisioning

A more annoying one: Grafana kept reporting “you do not currently have a default database configured for this data source,” even though our provisioning YAML clearly specified one. This took several rounds of trial and error (including a confusing detour through time-zone settings before we realized the dashboard panels were not the problem) before we traced it to how the current Grafana Postgres plugin expects the database name, which eventually required wiping Grafana’s data volume and re-provisioning cleanly. Once fixed, the dashboard has been stable since.

## 6.4 Demo runs performed so far

We have already run several of the demos we expect to repeat (or extend) for the final presentation:

- End-to-end run: producer → Kafka → Spark → Citus → Grafana dashboard, with live NORMAL/FLASH\_SALE cycling visible on the dashboard.
- Kafka fault tolerance: killing 1 of 3 brokers had no visible effect on the pipeline; killing 2 of 3 correctly caused the producer to fail with `NotEnoughReplicasError` rather than silently dropping data – i.e., our `RF=3 / min.insync.replicas=2` configuration enforces exactly the guarantee it was meant to, at exactly the boundary we expected.
- Citus fault tolerance: killing one of two workers mid-write caused a visible JDBC exception and, in one run, an actual crash and automatic restart of the Spark job (via Docker’s restart policy); Spark’s checkpointing meant no Kafka offsets were lost in the process.
- Spark scale-out: comparing 1 worker vs. 3 workers on similarly-sized batches showed roughly a 40–45% reduction in processing time at the same load.
- Citus scale-out: added a 3rd worker mid-project and ran Citus’s shard rebalancer, moving 8 of 32 shards per table onto the new node with no data loss.
- A 5x overload stress test, mostly out of curiosity about where things would actually break: the system did not crash, but instead settled into a steady state around 150–160k events per batch. The more interesting result was an unexpected one – once batches got that large, our simple per-batch anomaly threshold started flagging almost all traffic as “bot-like,” including normal users, simply because a large micro-batch naturally accumulates more events per IP than our fixed threshold assumed. We are planning to discuss this honestly as a limitation in the final report rather than hide it.

We are aware that this report already contains some results that the project brief technically associates with the final stage. We considered trimming them out to keep this checkpoint “proportionate,” but decided it was more useful to be transparent about where we actually are.

## 7. Remaining Work Toward the Final Report

- Write up the architecture, methodology, and results formally in the final report, with diagrams and the full set of measurements collected so far.
- Decide whether to add one more controlled experiment (e.g., a fix for the anomaly-threshold weakness found in Section 6.4) or simply discuss it as future work.
- Prepare the live demo (fault injection + Grafana walkthrough) for the 20-minute presentation slot.
- Clean up and finalize the submitted code/Docker Compose package so it can be reproduced from a clean machine.